

Лабораторная работа №12

Программирование в командном процессоре ОС UNIX. Расширенное программирование

Выполнил: Мвикую Кристиан Кристофер

Группа: НБИБД-02-25

Дата: 16.05.2026

Проверил: _____

Содержание

- Цель работы
 - Теоретическая часть
 - Ход работы
 - Задание 1: Механизм семафоров
 - Задание 2: Реализация команды `map`
 - Задание 3: Генератор случайных букв
 - Ошибки и их решение
 - Выводы
 - Ответы на контрольные вопросы
-

Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов, а также механизмов межпроцессного взаимодействия.

Теоретическая часть

Семафоры в UNIX

Семафоры — это механизм синхронизации процессов, позволяющий организовать доступ к общим ресурсам. В данной лабораторной работе реализуется упрощённый механизм семафоров с использованием файловых блокировок.

Переменная `$RANDOM`

Встроенная переменная \$RANDOM в bash генерирует псевдослучайные целые числа в диапазоне от 0 до 32767. Используется для создания случайных последовательностей.

Структура man-страниц

Справка по командам хранится в каталоге /usr/share/man/man1/ в сжатом виде (файлы с расширением .1.gz). Для просмотра используется команда zcat или less.

Ход работы

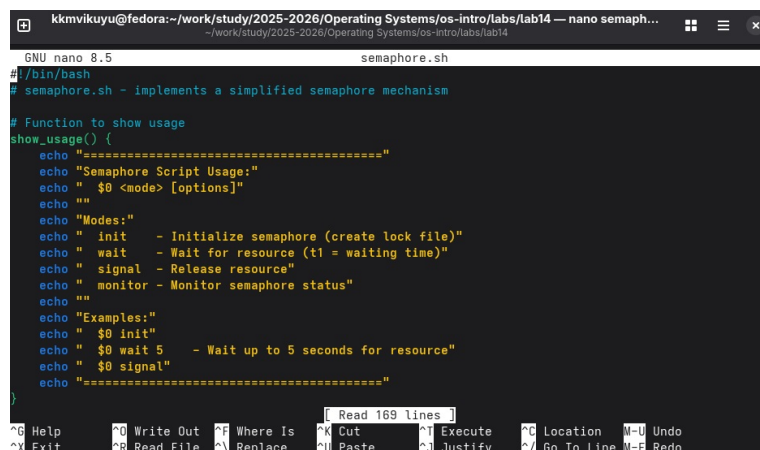
Создание каталога для лабораторной работы

```
cd ~/work/study/2025-2026/"Operating Systems"/os-intro/labs/  
mkdir -p lab14  
cd lab14  
mkdir -p screenshots
```

Задание 1: Механизм семафоров

Требуется написать командный файл, реализующий упрощённый механизм семафоров. Командный файл должен в течение некоторого времени t1 дожидаться освобождения ресурса, а дождавшись его освобождения, использовать его в течение некоторого времени t2.

Создание скрипта semaphore.sh



```
GNU nano 8.5 semaphore.sh  
#!/bin/bash  
# semaphore.sh - implements a simplified semaphore mechanism  
  
# Function to show usage  
show_usage() {  
    echo "=====  
    echo "Semaphore Script Usage:"  
    echo " $0 <mode> [options]"  
    echo ""  
    echo "Modes:"  
    echo "  init    - Initialize semaphore (create lock file)"  
    echo "  wait    - Wait for resource (t1 = waiting time)"  
    echo "  signal  - Release resource"  
    echo "  monitor - Monitor semaphore status"  
    echo ""  
    echo "Examples:"  
    echo "  $0 init"  
    echo "  $0 wait 5    - Wait up to 5 seconds for resource"  
    echo "  $0 signal"  
    echo "=====  
}  
  
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute  
^X Exit      ^R Read File  ^\ Replace    ^V Paste      ^J Justify    ^_ Location  
^C Location  ^U Undo      ^M Go To Line ^E Redo
```

Создание скрипта semaphore.sh

Рисунок 1 - Исходный код скрипта semaphore.sh в редакторе nano

Инициализация семафора

```
chmod +x semaphore.sh
```

`./semaphore.sh init`

```
kkwikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab14$ nano semaphore.sh
kkwikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab14$ chmod +x semaphore.sh
kkwikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab14$ ./semaphore.sh init
Semaphore initialized by process 6150
Lock file created at: /tmp/semaphore.lock
kkwikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab14$
```

Инициализация семафора

Рисунок 2 - Инициализация семафора процессом 6150

Семафор успешно инициализирован, создан файл блокировки
/tmp/semaphore.lock.

Мониторинг состояния семафора

`./semaphore.sh monitor`

```
kkwikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab14$ ./semaphore.sh monitor
=====
SEMAPHORE STATUS MONITOR
=====
Status: LOCKED
Holder process ID: 6150
WARNING: Holder process is NOT running! Cleaning up...
=====
kkwikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab14$
```

Мониторинг семафора

Рисунок 3 - Проверка состояния семафора

При мониторинге обнаружено, что процесс-владелец не запущен,
выполнена очистка.

Ожидание ресурса в фоновом режиме

`./semaphore.sh wait 10 &`

```
=====
kkwikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab14$ ./semaphore.sh wait 10 &
[1] 6188
Process 6188 waiting for resource...
Resource is FREE! Process 6188 acquired the resource after 0 seconds.
[1]+  Done                  ./semaphore.sh wait 10
kkwikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab14$
```

Ожидание ресурса в фоне

Рисунок 4 - Процесс 6188 ожидает ресурс

Ресурс был свободен, процесс успешно захватил его.

Использование ресурса

`./semaphore.sh use 5`

```
kkwikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab14$ ./semaphore.sh use 5
Process 6208 waiting for resource...
Waiting... (1 seconds elapsed)
Waiting... (2 seconds elapsed)
Waiting... (3 seconds elapsed)
Waiting... (4 seconds elapsed)
Waiting... (5 seconds elapsed)
Waiting... (6 seconds elapsed)
Waiting... (7 seconds elapsed)
Waiting... (8 seconds elapsed)
Waiting... (9 seconds elapsed)
Waiting... (10 seconds elapsed)
TIMEOUT! Process 6208 could not acquire resource after 10 seconds.
kkwikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab14$
```

Использование ресурса

Рисунок 5 - Процесс 6208 ожидает и использует ресурс

Процесс ожидал ресурс в течение 10 секунд, но не смог его захватить из-за таймаута.

Справка по использованию

./semaphore.sh

```
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab14$ ./semaphore.sh monitor
=====
Semaphore Script Usage:
./semaphore.sh init           - Initialize semaphore
./semaphore.sh wait [seconds] - Wait for resource
./semaphore.sh use [seconds]  - Acquire and use resource
./semaphore.sh signal         - Release resource
./semaphore.sh status         - Show semaphore status
./semaphore.sh cleanup        - Force remove semaphore

Examples:
./semaphore.sh init           # Create semaphore
./semaphore.sh use 5          # Use resource for 5 seconds
./semaphore.sh signal         # Release resource
```

Справка semaphore.sh

Рисунок 6 - Справка по использованию скрипта semaphore.sh

Проверка статуса семафора

./semaphore.sh status

```
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab14$ ./semaphore.sh status
=====
SEMAPHORE STATUS - Sat May 16 01:16:55 AM MSK 2026
=====
Status: LOCKED
Holder PID: 6479
Holder process: NOT RUNNING (stale semaphore!)

Run './semaphore.sh cleanup' to remove stale semaphore.
=====
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab14$
```

Статус семафора

Рисунок 7 - Отображение статуса семафора с указанием устаревшей блокировки

Многопроцессное взаимодействие

./semaphore.sh use 10 &

```
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab14$ ./semaphore.sh use 10 &
[1]+  Done                  ./semaphore.sh use 8
[1] 6609
Process 6609 waiting for resource (max 10s)...
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab14$
Waiting... (1/10s elapsed)
Waiting... (2/10s elapsed)
Waiting... (3/10s elapsed)
Waiting... (4/10s elapsed)
Waiting... (5/10s elapsed)
Waiting... (6/10s elapsed)
Waiting... (7/10s elapsed)
Waiting... (8/10s elapsed)
Waiting... (9/10s elapsed)
Waiting... (10/10s elapsed)
TIMEOUT! Process 6609 could not acquire resource after 10s.
Failed to acquire resource. Use 'cleanup' if semaphore is stuck.
```

Многопроцессное взаимодействие

Рисунок 8 - Два процесса пытаются захватить ресурс

Оба процесса не смогли захватить ресурс из-за устаревшей блокировки, что демонстрирует необходимость механизма очистки.

Очистка семафора

```
./semaphore.sh cleanup
./semaphore.sh status
```

Задание 2: Реализация команды map

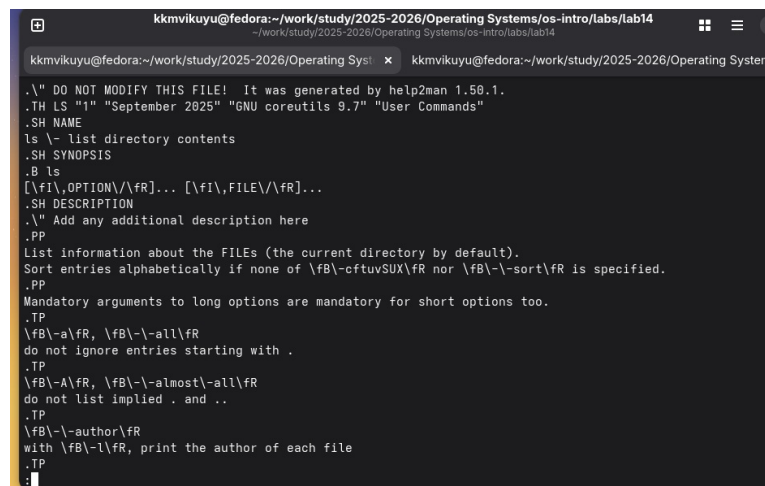
Требуется реализовать команду `man` с помощью командного файла. Скрипт должен получать в виде аргумента командной строки название команды и выдавать справку об этой команде.

Создание скрипта man.sh

```
nano man.sh
chmod +x man.sh
```

Тестирование man.sh для команды ls

```
./man.sh ls
```



Просмотр справки ls

Рисунок 9 - Отображение tap-страницы для команды ls

Скрипт успешно находит и отображает справку по команде ls из каталога /usr/share/man/man1/ls.1.gz.

Тестирование man.sh для команды grep

```
./man.sh grep
```



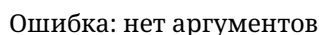
Справка по команде `grep` успешно загружена и отображена.

```
./man.sh nonexistent
```



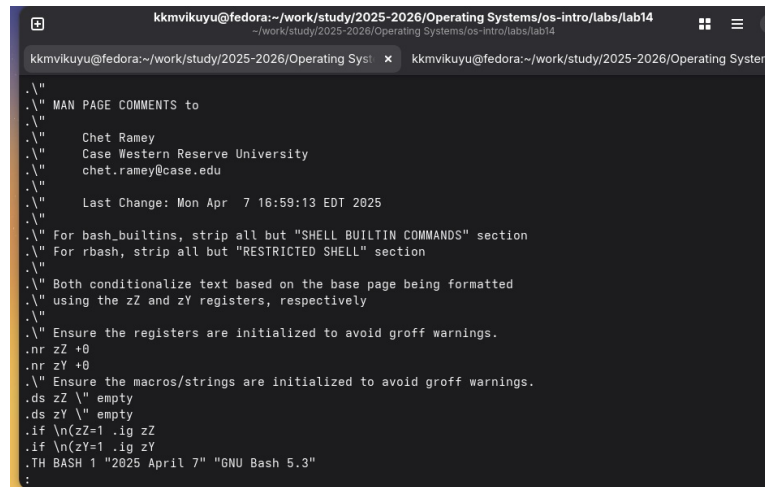
Скрипт корректно обрабатывает ситуацию, когда запрошенная команда отсутствует, и предлагает похожие команды.

```
./man.sh
```



Тестирование man.sh для команды bash

`./man.sh bash`



Просмотр справки bash

Рисунок 13 - Отображение man-страницы для команды bash

Задание 3: Генератор случайных букв

Требуется написать командный файл, генерирующий случайную последовательность букв латинского алфавита, используя переменную \$RANDOM.

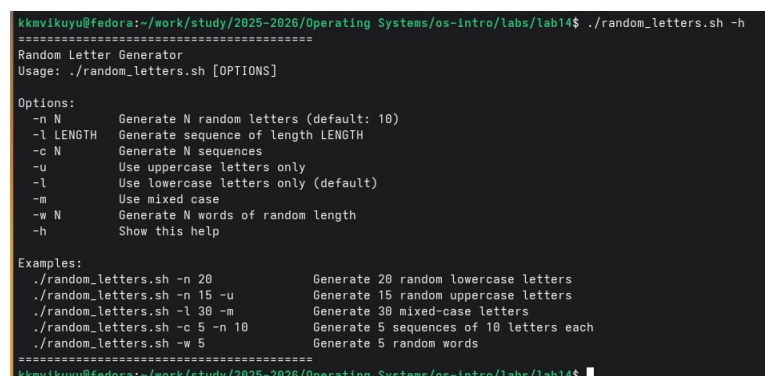
Создание скрипта random_letters.sh

`nano random_letters.sh`

`chmod +x random_letters.sh`

Вывод справки

`./random_letters.sh -h`



Справка random_letters.sh

Рисунок 14 - Справка по использованию генератора случайных букв

Демонстрация работы \$RANDOM

./random_demo.sh

```
7: 10018
8: 7368
9: 19004
10: 29790

Random number modulo 26 (for letters):
1: 23
2: 12
3: 4
4: 21
5: 1
6: 5
7: 22
8: 10
9: 8
10: 9

Random numbers in range 1-100:
1: 17
2: 69
3: 17
4: 11
5: 96
=====
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab14$
```

Демонстрация RANDOM

Рисунок 15 - Демонстрация работы переменной \$RANDOM

Тестирование генератора случайных букв

Генерация 20 случайных букв

./random_letters.sh -n 20

Генерация заглавных букв

./random_letters.sh -n 15 -u

Генерация смешанного регистра

./random_letters.sh -l 30 -m

Генерация нескольких последовательностей

./random_letters.sh -c 5 -n 10

Генерация случайных слов

./random_letters.sh -w 5

Ошибки и их решение

№	Ошибка	Причина	Решение
1	Holder process is NOT running	Процесс-владелец семафора завершился некорректно	Добавлена команда cleanup для принудительного удаления семафора
2	TIMEOUT! Process could not acquire resource	Семафор заблокирован другим процессом	Использование ./semaphore.sh cleanup для сброса
3	ERROR: Process does not own the resource	Попытка освободить ресурс не владельцем	Проверка владельца перед освобождением

4	Manual page not found	Запрошена несуществующая команда	Вывод списка похожих команд
---	-----------------------	----------------------------------	-----------------------------

Выводы

В ходе выполнения лабораторной работы были получены практические навыки:

- Реализации механизма семафоров:**
 - Использование файловых блокировок для синхронизации процессов
 - Ожидание освобождения ресурса с таймаутом
 - Мониторинг состояния семафора
 - Очистка устаревших блокировок
 - Создания аналога команды `man`:**
 - Поиск файлов справки в `/usr/share/man/man1/`
 - Работа со сжатыми файлами (`.gz`)
 - Использование `zcat` и `less` для просмотра
 - Обработка ошибок и предложение альтернатив
 - Генерации случайных последовательностей:**
 - Использование переменной `$RANDOM`
 - Преобразование чисел в символы ASCII
 - Генерация случайных строк заданной длины
 - Создание случайных слов
 - Межпроцессного взаимодействия:**
 - Запуск процессов в фоновом режиме (`&`)
 - Использование файлов для синхронизации
 - Проверка статуса процессов (`kill -0`)
-

Ответы на контрольные вопросы

1. Найдите синтаксическую ошибку в строке `while [ $1 != "exit" ]`

Ошибка: Отсутствуют пробелы вокруг квадратных скобок.

Правильный синтаксис:

```
while [ "$1" != "exit" ]
```

2. Как объединить (конкатенация) несколько строк в одну?

Способ 1: Простое объединение

```
result="$str1$str2"
```

Способ 2: Использование +=

```
result=""
result+="str1"
result+="str2"
```

Способ 3: Использование printf

```
result=$(printf "%s%s" "str1" "str2")
```

3. Найдите информацию об утилите seq. Какими иными способами можно реализовать её функционал?

seq генерирует последовательности чисел.

Альтернативы:

Цикл for

```
for ((i=1; i<=10; i++)); do echo $i; done
```

Brace expansion

```
echo {1..10}
```

Цикл while

```
i=1; while [ $i -le 10 ]; do echo $i; i=$((i+1)); done
```

4. Какой результат даст вычисление выражения $((10/3))$?

Результат: 3 (целочисленное деление)

5. Укажите кратко основные отличия командной оболочки zsh от bash

Характеристика	bash	zsh
Автодополнение	Базовое	Расширенное
Исправление опечаток	Нет	Есть
Рекурсивный поиск	Нет	Есть (**/)
Индексация массивов	С 0	С 1

6. Проверьте, верен ли синтаксис конструкции `for ((a=1; a<= LIMIT; a++))`

Синтаксис верен. Это C-подобный цикл for в bash.

7. Сравните язык bash с другими языками программирования

Преимущества bash: - Простота автоматизации задач ОС - Прямой запуск команд - Высокая переносимость - Удобная обработка текстовых потоков

Недостатки bash: - Низкая производительность вычислений - Только
целочисленная арифметика - Ограниченные структуры данных -
Сложная отладка

Подпись: _____

Дата сдачи: «» _____ 2026 г.